

Assignment 1

Sinh viên: Luru Anh Dũng

Mã sinh viên: B23DCDK036

1. Introduction

This report presents a small decision-support system for early diabetes screening, using the Pima Indians Diabetes dataset. The goal is to walk through the full pipeline of an intelligent system end to end: starting from raw clinical measurements, choosing how to represent them, training and comparing several classical machine learning models, and finally wiring the best one into a small demo application.

The report is organized as follows. Sections 2 and 3 define the system and the learning problem it solves. Sections 4 and 5 describe the dataset and how patient records are turned into feature vectors. Section 6 introduces the machine learning methods used. Section 7 explains the experimental setup, and Section 8 reports the training results for each model, including exploratory data analysis and a look at how sensitive k-NN is to its hyperparameters. Section 9 compares all models directly, Section 10 checks how much feature scaling actually matters, and Section 11 shows the trained model running inside a demo application. The report closes with a discussion of limitations (Section 12), a reflection on the project (Section 13), and a final conclusion (Section 14).

2. Intelligent System Definition

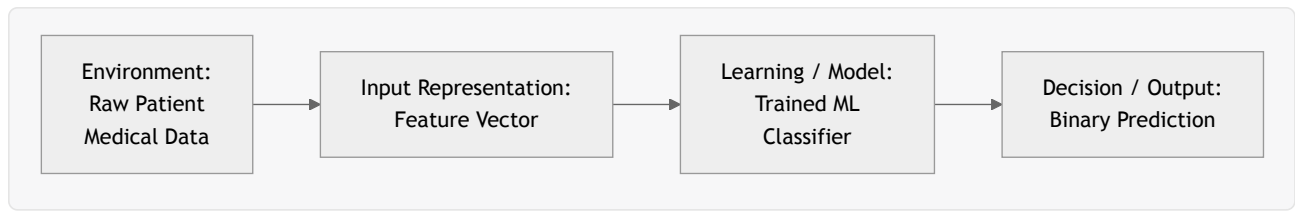
This system acts as a clinical decision-support tool for the early triage of patients at risk of diabetes. It looks at a patient's demographic and clinical measurements and returns a prediction, which a healthcare professional can use to decide who needs closer follow-up.

System characteristics:

1. **Real-world problem.** Diabetes is easier to manage the earlier it is caught. The system uses routine clinical measurements that are already collected during a checkup to flag patients who may be at higher risk, before more serious complications develop.
2. **System input.** The environment gives the system a patient's age together with seven clinical measurements: Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, and Diabetes Pedigree Function.
3. **Internal representation.** These raw values are represented internally as an 8-dimensional numerical feature vector. Missing or physiologically impossible zero readings are filled in using statistics from the training data, and the vector is standardized when the learning algorithm is distance-based.
4. **Learning objective.** The model learns a decision boundary that maps this 8-dimensional representation to the diagnostic outcomes observed in the training data.
5. **System output.** The system returns a binary decision — diabetic (1) or non-diabetic (0) — along with a probability estimate of the risk.
6. **Application context.** The intended users are clinicians doing an initial screening, and patients using the companion demo app to get a rough sense of their own risk.

Formal problem statement: *Given an 8-dimensional feature vector describing a patient's clinical profile, predict whether that patient is diabetic or non-diabetic.*

Pipeline diagram:



3. Problem Definition

Learning task. Supervised binary classification. The goal is to learn a function $f_{\theta}(x)$ that predicts whether `Outcome` is 1 (diabetic) or 0 (non-diabetic) given a patient's feature vector.

Feature space. $x \in \mathbb{R}^8$, made up of eight numerical features: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, and Age.

Target output. $\hat{y} \in \{0, 1\}$, the predicted class, together with a class probability so results can be used for risk-based triage rather than a hard yes/no.

Formal problem statement: Given an 8-dimensional feature vector describing a patient's clinical profile, predict whether that patient is diabetic or non-diabetic.

4. Dataset

This project uses the **Diabetes Dataset** from Kaggle. Source: Kaggle — hasibur013/diabetes-dataset

Dataset Description

1. **What it represents.** Physiological and demographic indicators linked to the onset of diabetes, collected from female patients of Pima Indian heritage.
2. **Observation.** Each row is the clinical profile of one patient recorded during a medical visit.
3. **Features.** Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, and Age.
4. **Target variable.** `Outcome` — the clinical diabetes diagnosis.
5. **Target type.** Categorical, binary.
6. **Task type.** Classification.
7. **Number of observations.** 768 patient records.
8. **Number of features.** 8 input features.
9. **Numerical features.** All 8 features are numerical — a mix of continuous measurements and integer counts.
10. **Categorical features.** None; every input feature is numerical.

5. Data Representation

A patient's clinical profile is represented as an 8-dimensional feature vector: $x_i = [x_{i1}, x_{i2}, \dots, x_{i8}] \in \mathbb{R}^8$.

Feature	Type	Representation	Meaning
Pregnancies	Numerical	Integer	Number of pregnancies
Glucose	Numerical	Real value	Plasma glucose concentration
BloodPressure	Numerical	Real value	Diastolic blood pressure (mm Hg)
SkinThickness	Numerical	Real value	Triceps skin-fold thickness (mm)
Insulin	Numerical	Real value	2-hour serum insulin (mu U/ml)
BMI	Numerical	Real value	Body mass index
DiabetesPedigreeFunction	Numerical	Real value	Pedigree-based diabetes likelihood score
Age	Numerical	Integer	Age in years

Preprocessing. Even though every feature starts out numerical, the raw values still need some cleanup before they are model-ready. A reading of zero in a field like Glucose or BMI is not physiologically possible, so these are treated as missing and imputed with the median computed from the training set. Features are also standardized to zero mean and unit variance, so that variables with naturally large ranges (like Insulin) don't end up dominating distance-based models simply because of their scale. The final input to each model is therefore this cleaned and standardized version of the raw data, not the raw values themselves.

Feature and Target Analysis

All input features are numerical, and the target is a binary category. The features sit on very different scales — DiabetesPedigreeFunction values are usually under 3, while Insulin readings can run into the hundreds. This gap matters for distance-based models such as k-Nearest Neighbors and Support Vector Machines, since they compare feature vectors using geometric distance. Without scaling, a feature like Insulin would dominate that distance purely because of its larger numeric range, not because it is more informative — which is why standardization is applied before training these models.

6. Traditional ML Methods

Several classical machine learning classifiers are trained and compared. A simple baseline is included first to give a lower bound, followed by six models tuned with cross-validated grid search.

- **Baseline (`DummyClassifier`):** Always predicts the majority class (Non-Diabetic). It sets the minimum bar that any real model needs to clear.
- **Logistic Regression:** A linear model that estimates the log-odds of the positive class as a weighted sum of the input features.
- **Support Vector Machine (Linear):** Finds the hyperplane that separates the two classes with the largest possible margin.
- **Support Vector Machine (RBF):** Uses a Radial Basis Function kernel to project the data into a higher-dimensional space, which lets it capture non-linear decision boundaries.
- **k-Nearest Neighbors (k-NN):** Classifies a new patient by majority vote among its `k` closest neighbors in the standardized feature space.
- **Random Forest:** An ensemble of decision trees, each trained on a bootstrap sample of the data, combined to reduce variance and overfitting.
- **XGBoost:** A gradient boosting method that builds trees sequentially, with each new tree correcting the errors left by the ones before it.

7. Experimental Design

The dataset is split into training and test sets with an 80/20 ratio, using stratified sampling on `Outcome` so both sets keep roughly the same class balance.

This split happens before any imputation or scaling. All preprocessing statistics — the median used for imputation, the mean and variance used for standardization — are computed only from the training set, then applied to both training and test sets. Doing it in this order keeps the test set genuinely unseen during preprocessing and avoids leaking any information from it into training.

```

import warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

# Silence noisy-but-benign library warnings (e.g. the baseline's undefined
# precision on a class it never predicts, and sklearn's SVC `probability`
# deprecation notice) so the report output stays focused on real results.
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, auc,
)

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
import xgboost as xgb

sns.set_theme(style="whitegrid", palette="deep")

# 1. Load the raw clinical records
df = pd.read_csv("diabetes.csv")
df.columns = df.columns.str.strip()

# 2. Split off the test set BEFORE any imputation/scaling statistic is
#     computed, so nothing about it can bias preprocessing decisions.
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
X_train, X_test = X_train.copy(), X_test.copy()

# 3. Treat physiologically-invalid zeros as missing, then impute using
#     statistics learned from the TRAINING split only.
cols_to_impute = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
X_train[cols_to_impute] = X_train[cols_to_impute].replace(0, np.nan)
X_test[cols_to_impute] = X_test[cols_to_impute].replace(0, np.nan)

imputer = SimpleImputer(strategy='median')
X_train[cols_to_impute] = imputer.fit_transform(X_train[cols_to_impute])
X_test[cols_to_impute] = imputer.transform(X_test[cols_to_impute])
joblib.dump(imputer, 'imputer.pkl')

# 4. Feature scaling (fit on the training split only)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
joblib.dump(scaler, 'scaler.pkl')

```

```

# 5. Shared bookkeeping and a reusable scoring/plotting helper for every
#     model trained below.
model_metrics = []
model_filenames = {}
model_needs_scaling = {}
model_objects = {}

def evaluate_model(name, model, X_te, y_te, filename, needs_scaling):
    """Score a fitted model, log it for the comparison table in Section 9,
    persist it to disk, and render its confusion matrix."""
    y_pred = model.predict(X_te)
    acc = accuracy_score(y_te, y_pred)
    prec = precision_score(y_te, y_pred, zero_division=0)
    rec = recall_score(y_te, y_pred, zero_division=0)
    f1 = f1_score(y_te, y_pred, zero_division=0)

    model_metrics.append({'Model': name, 'Accuracy': acc, 'Precision': prec, 'Recall': rec, 'F1': f1})
    model_filenames[name] = filename
    model_needs_scaling[name] = needs_scaling
    model_objects[name] = model
    joblib.dump(model, filename)

    print(f"Accuracy: {acc*100:.2f}% | Precision: {prec:.3f} | Recall: {rec:.3f} | F1: {f1:.3f}")
    print(classification_report(y_te, y_pred, target_names=['Non-Diabetic', 'Diabetic']))

    cm = confusion_matrix(y_te, y_pred)
    cm_pct = cm / cm.sum() * 100
    labels = np.array([f"{count}\n({pct:.1f}%" for count, pct in zip(cm.flatten(), cm_pct.flatten())])
    plt.figure(figsize=(4.5, 4))
    sns.heatmap(cm, annot=labels, fmt='', cmap='YlGnBu', cbar=False,
                xticklabels=['Non-Diabetic', 'Diabetic'], yticklabels=['Non-Diabetic', 'Diabetic'])
    plt.title(f'{name} - Confusion Matrix')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.tight_layout()
    plt.show()
    return y_pred

```

8. Results

This section reports what happened when each model was trained and evaluated: the baseline and the six tuned classifiers from Section 6, an exploratory look at the data itself, and a closer look at how sensitive k-NN is to its hyperparameters.

Evaluation Protocol

Every trained model, including the baseline, is scored on the test set using Accuracy, Precision, Recall, F1-score, and a confusion matrix.

In a screening context, **Recall** matters most: missing an actual diabetic patient (a false negative) is a far more costly mistake than flagging a healthy patient for a second look (a false positive). The dataset is also moderately imbalanced toward non-diabetic cases, so Accuracy on its own can be misleading. For this reason, the **F1-score** — which balances Precision and Recall — is used as the main metric for comparing models and picking the final one.

Baseline

A `DummyClassifier` using the `most_frequent` strategy provides the baseline.

Why bother with a baseline? It shows what "no real learning at all" looks like, so it's possible to check that the trained models are actually picking up a useful signal from the data rather than just reflecting the class imbalance. Any model that can't beat this baseline isn't learning anything useful.

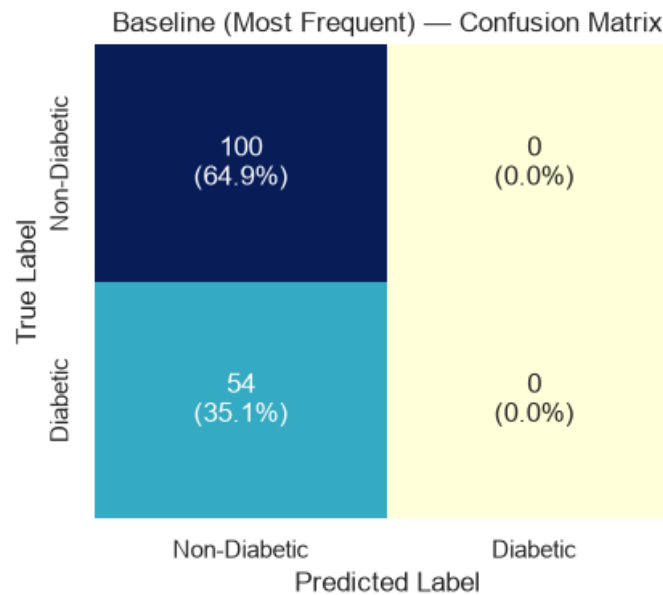
```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy='most_frequent')
dummy.fit(X_train, y_train)
y_pred_dummy = evaluate_model('Baseline (Most Frequent)', dummy, X_test, y_test, 'baseline.pkl', needs
```

```
Accuracy: 64.94% | Precision: 0.000 | Recall: 0.000 | F1: 0.000
      precision    recall  f1-score   support
```

```
Non-Diabetic      0.65      1.00      0.79      100
   Diabetic       0.00      0.00      0.00       54

   accuracy                0.65      154
  macro avg      0.32      0.50      0.39      154
weighted avg      0.42      0.65      0.51      154
```



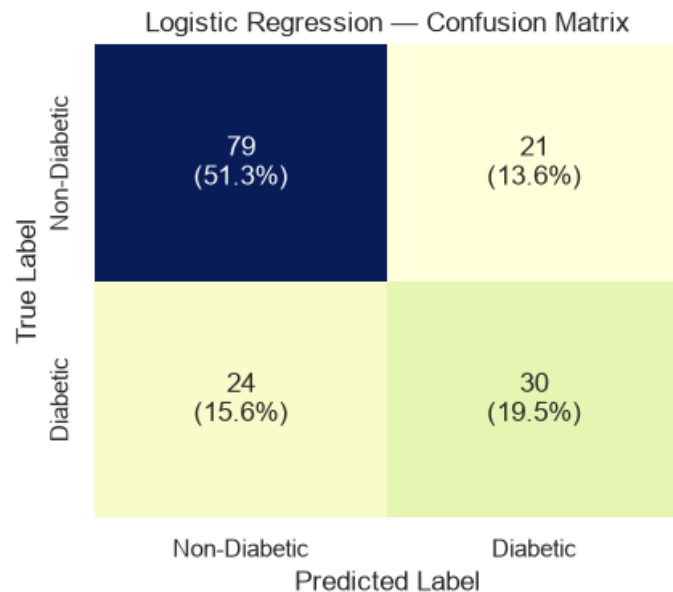
Model 1: Logistic Regression

```
param_grid_lr = {'C': [0.01, 0.1, 1, 10, 100]}
grid_lr = GridSearchCV(LogisticRegression(random_state=42, max_iter=1000), param_grid_lr, cv=5, scoring='accuracy')
grid_lr.fit(X_train_scaled, y_train)
best_lr = grid_lr.best_estimator_
print(f"Best hyperparameters: {grid_lr.best_params_}")

y_pred_lr = evaluate_model('Logistic Regression', best_lr, X_test_scaled, y_test, 'logistic_regression')
```

```
Best hyperparameters: {'C': 0.1}
Accuracy: 70.78% | Precision: 0.588 | Recall: 0.556 | F1: 0.571
```

	precision	recall	f1-score	support
Non-Diabetic	0.77	0.79	0.78	100
Diabetic	0.59	0.56	0.57	54
accuracy			0.71	154
macro avg	0.68	0.67	0.67	154
weighted avg	0.70	0.71	0.71	154

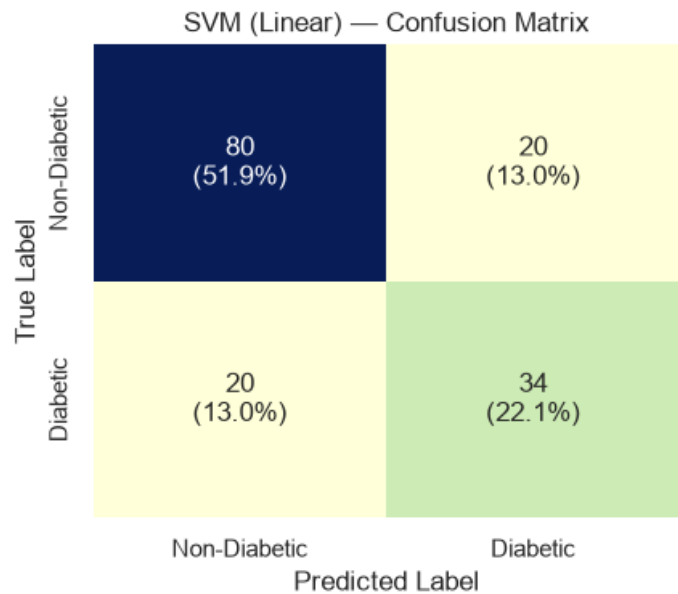


Model 2: SVM (Linear)

```
param_grid_svml = {'C': [0.01, 0.1, 1, 10]}
grid_svml = GridSearchCV(SVC(kernel='linear', random_state=42, probability=True), param_grid_svml, cv=
grid_svml.fit(X_train_scaled, y_train)
best_svml = grid_svml.best_estimator_
print(f"Best hyperparameters: {grid_svml.best_params_}")

y_pred_svml = evaluate_model('SVM (Linear)', best_svml, X_test_scaled, y_test, 'svm_linear.pkl', needs
```

Best hyperparameters: {'C': 1}				
Accuracy: 74.03% Precision: 0.630 Recall: 0.630 F1: 0.630				
	precision	recall	f1-score	support
Non-Diabetic	0.80	0.80	0.80	100
Diabetic	0.63	0.63	0.63	54
accuracy			0.74	154
macro avg	0.71	0.71	0.71	154
weighted avg	0.74	0.74	0.74	154



Model 3: SVM (RBF)

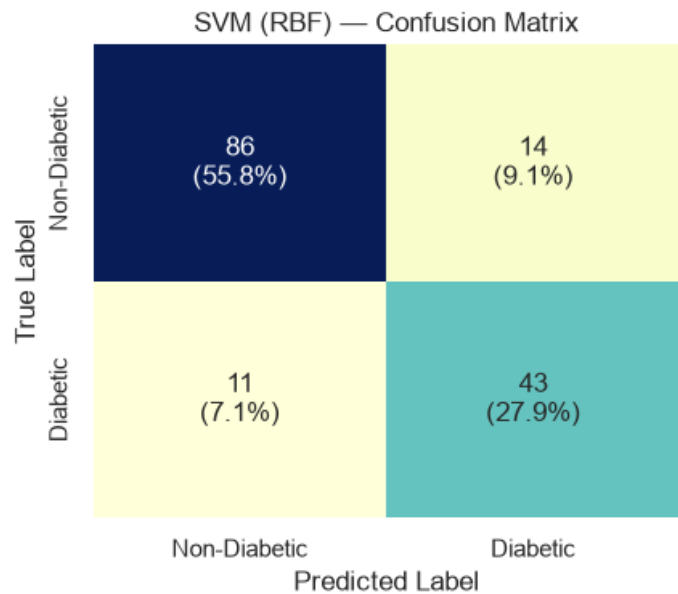
```
param_grid_svmr = {'C': [0.1, 1, 10], 'gamma': ['scale', 'auto', 0.01, 0.1]}
grid_svmr = GridSearchCV(SVC(kernel='rbf', random_state=42, probability=True), param_grid_svmr, cv=5,
grid_svmr.fit(X_train_scaled, y_train)
best_svmr = grid_svmr.best_estimator_
print(f"Best hyperparameters: {grid_svmr.best_params_}")

y_pred_svmr = evaluate_model('SVM (RBF)', best_svmr, X_test_scaled, y_test, 'svm_rbf.pkl', needs_scali
```

Best hyperparameters: {'C': 1, 'gamma': 0.1}

Accuracy: 83.77% | Precision: 0.754 | Recall: 0.796 | F1: 0.775

	precision	recall	f1-score	support
Non-Diabetic	0.89	0.86	0.87	100
Diabetic	0.75	0.80	0.77	54
accuracy			0.84	154
macro avg	0.82	0.83	0.82	154
weighted avg	0.84	0.84	0.84	154



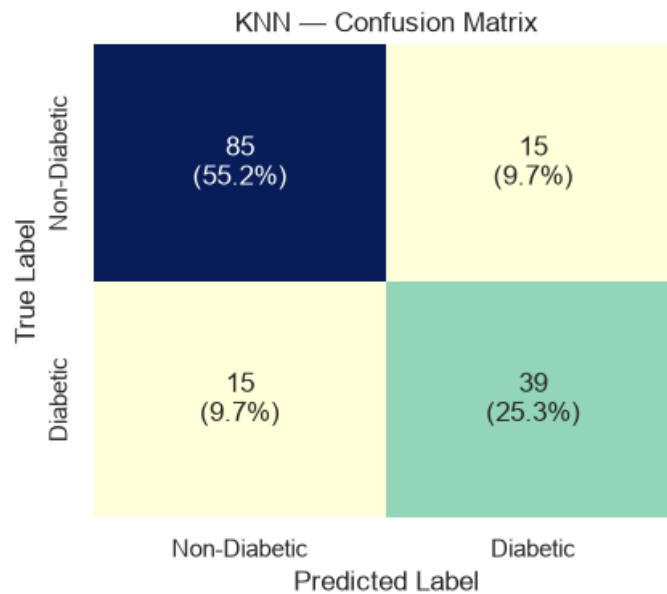
Model 4: KNN

```
param_grid_knn = {'n_neighbors': [3, 5, 7, 9], 'weights': ['uniform', 'distance']}
grid_knn = GridSearchCV(KNeighborsClassifier(), param_grid_knn, cv=5, scoring='accuracy')
grid_knn.fit(X_train_scaled, y_train)
best_knn = grid_knn.best_estimator_
print(f"Best hyperparameters: {grid_knn.best_params_}")

y_pred_knn = evaluate_model('KNN', best_knn, X_test_scaled, y_test, 'knn.pkl', needs_scaling=True)
```

```
Best hyperparameters: {'n_neighbors': 7, 'weights': 'uniform'}
Accuracy: 80.52% | Precision: 0.722 | Recall: 0.722 | F1: 0.722
```

	precision	recall	f1-score	support
Non-Diabetic	0.85	0.85	0.85	100
Diabetic	0.72	0.72	0.72	54
accuracy			0.81	154
macro avg	0.79	0.79	0.79	154
weighted avg	0.81	0.81	0.81	154



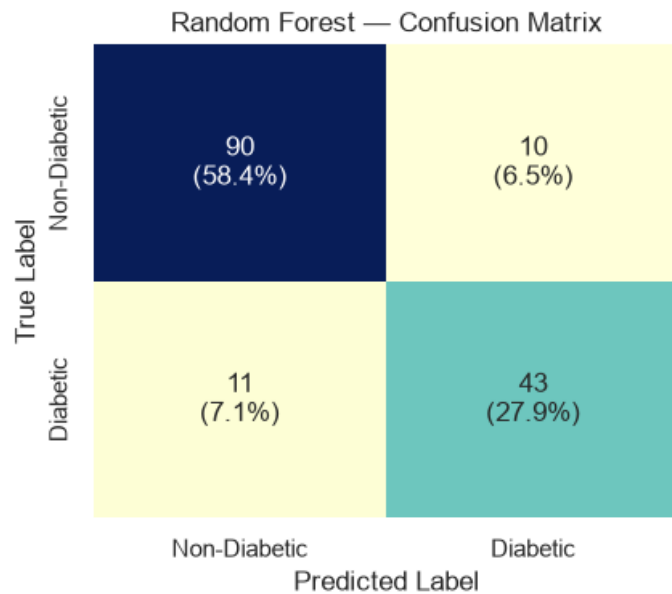
Model 5: Random Forest

```
param_grid_rf = {'n_estimators': [50, 100, 200], 'max_depth': [None, 5, 10]}
grid_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid_rf, cv=5, scoring='accuracy')
grid_rf.fit(X_train, y_train)
best_rf = grid_rf.best_estimator_
print(f"Best hyperparameters: {grid_rf.best_params_}")

y_pred_rf = evaluate_model('Random Forest', best_rf, X_test, y_test, 'random_forest.pkl', needs_scalin
```

```
Best hyperparameters: {'max_depth': 10, 'n_estimators': 200}
Accuracy: 86.36% | Precision: 0.811 | Recall: 0.796 | F1: 0.804
```

	precision	recall	f1-score	support
Non-Diabetic	0.89	0.90	0.90	100
Diabetic	0.81	0.80	0.80	54
accuracy			0.86	154
macro avg	0.85	0.85	0.85	154
weighted avg	0.86	0.86	0.86	154



Model 6: XGBoost

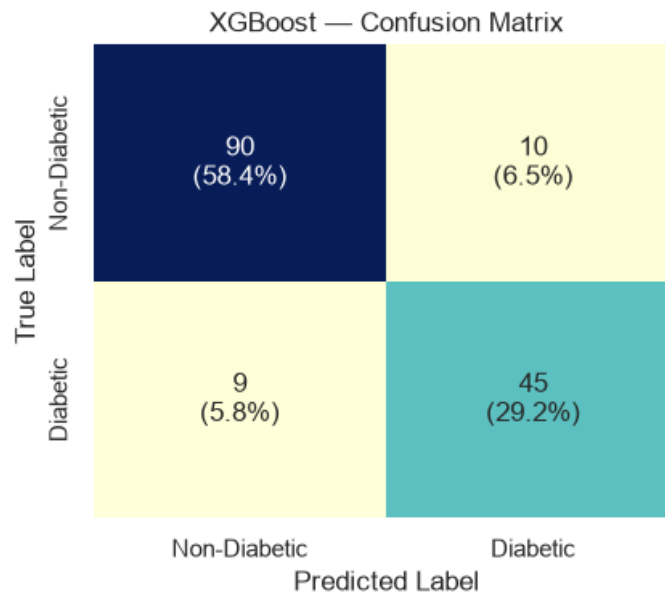
```
param_grid_xgb = {'n_estimators': [50, 100, 200], 'learning_rate': [0.01, 0.1, 0.2], 'max_depth': [3,
grid_xgb = GridSearchCV(xgb.XGBClassifier(eval_metric='logloss', random_state=42), param_grid_xgb, cv=
grid_xgb.fit(X_train, y_train)
best_xgb = grid_xgb.best_estimator_
print(f"Best hyperparameters: {grid_xgb.best_params_}")

y_pred_xgb = evaluate_model('XGBoost', best_xgb, X_test, y_test, 'xgboost.pkl', needs_scaling=False)
```

Best hyperparameters: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 50}

Accuracy: 87.66% | Precision: 0.818 | Recall: 0.833 | F1: 0.826

	precision	recall	f1-score	support
Non-Diabetic	0.91	0.90	0.90	100
Diabetic	0.82	0.83	0.83	54
accuracy			0.88	154
macro avg	0.86	0.87	0.87	154
weighted avg	0.88	0.88	0.88	154



Exploratory Data Analysis

Before looking at model outputs, it's worth understanding the data itself — how the features are distributed, how they relate to each other, and how they differ between diabetic and non-diabetic patients.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("diabetes.csv")
df.columns = df.columns.str.strip()

display(df.head())
df.info()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72.0	35	169.5	33.6	0.627	50	1
1	1	85	66.0	29	102.5	26.6	0.351	31	0
2	8	183	64.0	32	169.5	23.3	0.672	32	1
3	1	89	66.0	23	94.0	28.1	0.167	21	0
4	0	137	40.0	35	168.0	43.1	2.288	33	1

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Pregnancies                          768 non-null    int64
1   Glucose                              768 non-null    int64
2   BloodPressure                        768 non-null    float64
3   SkinThickness                        768 non-null    int64
4   Insulin                              768 non-null    float64
5   BMI                                  768 non-null    float64
6   DiabetesPedigreeFunction             768 non-null    float64
7   Age                                  768 non-null    int64
8   Outcome                              768 non-null    int64
dtypes: float64(4), int64(5)
memory usage: 54.1 KB
```

Feature Correlation Structure

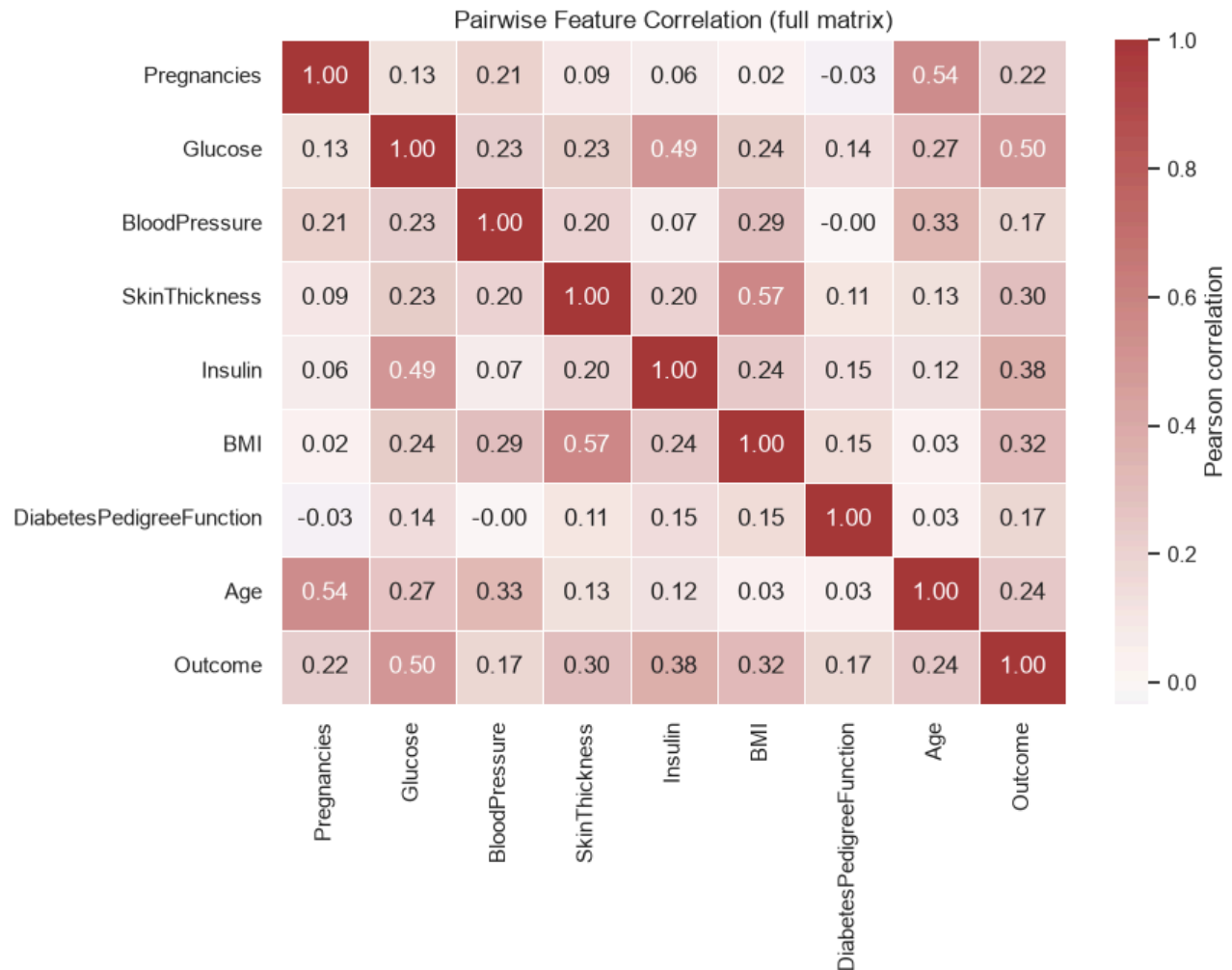
- **Visualization:** A heatmap detailing the Pearson correlation coefficients across all numeric features.
- **Analytical Purpose:** This analysis identifies multicollinearity and highlights features that exhibit strong linear relationships with the target variable, providing insight into the predictive signals available to the models.

```

corr = df.corr(numeric_only=True)

plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt=".2f", cmap='vlag', center=0,
            linewidths=0.5, cbar_kws={'label': 'Pearson correlation'})
plt.title('Pairwise Feature Correlation (full matrix)')
plt.tight_layout()
plt.show()

```



Correlation with the Target Variable

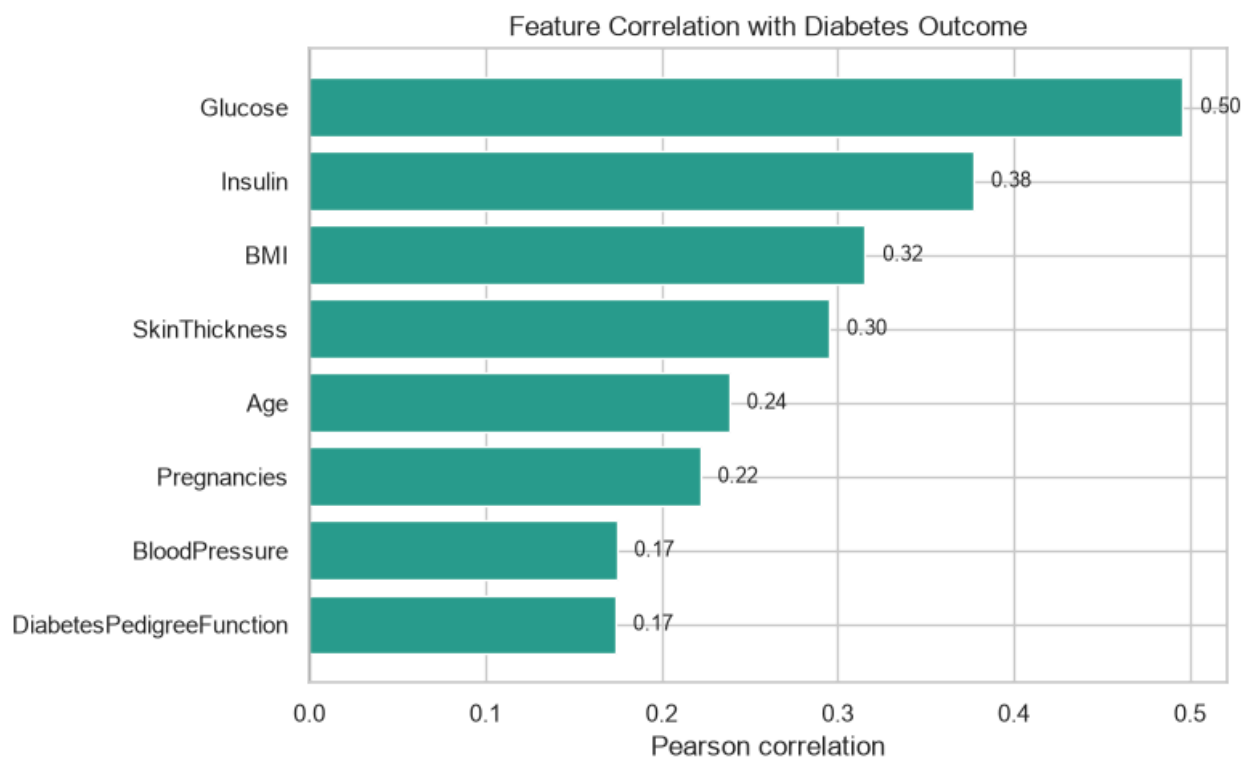
- **Visualization:** A ranked bar chart of Pearson correlation coefficients between individual features and the `Outcome`.
- **Analytical Purpose:** This provides a clear, univariate ranking of feature importance based on linear association with the diagnostic outcome.

```

outcome_corr = df.corr(numeric_only=True)[ 'Outcome' ].drop( 'Outcome' ).sort_values()
colors = [ '#c1121f' if v < 0 else '#2a9d8f' for v in outcome_corr.values]

plt.figure(figsize=(8, 5))
bars = plt.barh(outcome_corr.index, outcome_corr.values, color=colors)
for bar, v in zip(bars, outcome_corr.values):
    plt.text(v + (0.01 if v >= 0 else -0.01), bar.get_y() + bar.get_height() / 2,
             f"{v:.2f}", va='center', ha='left' if v >= 0 else 'right', fontsize=9)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Feature Correlation with Diabetes Outcome')
plt.xlabel('Pearson correlation')
plt.tight_layout()
plt.show()

```



Pairwise Feature Relationships

- **Visualization:** A pairwise scatterplot matrix for key predictors (Glucose, BMI, Age, and Insulin), segregated by class outcome.
- **Analytical Purpose:** This multidimensional view helps identify complex, non-linear class separations that may not be apparent in univariate analyses.

```

features_to_plot = ['Glucose', 'BMI', 'Age', 'Insulin', 'Outcome']

g = sns.pairplot(df[features_to_plot], hue='Outcome', corner=True,
                 palette={0: '#2a9d8f', 1: '#e76f51'}, diag_kind='hist',
                 plot_kws={'alpha': 0.6, 's': 25})
g.fig.suptitle('Pairwise Relationships by Outcome (lower triangle)', y=1.02)
plt.show()

```

Pairwise Relationships by Outcome (lower triangle)



Distribution Shift by Outcome

- **Visualization:** Box-and-stripe plots illustrating the distribution of key clinical measurements for both diabetic and non-diabetic groups.
- **Analytical Purpose:** These plots contrast the central tendencies and variances between the two classes, revealing the degree to which specific physiological measurements diverge based on the diabetes diagnosis.

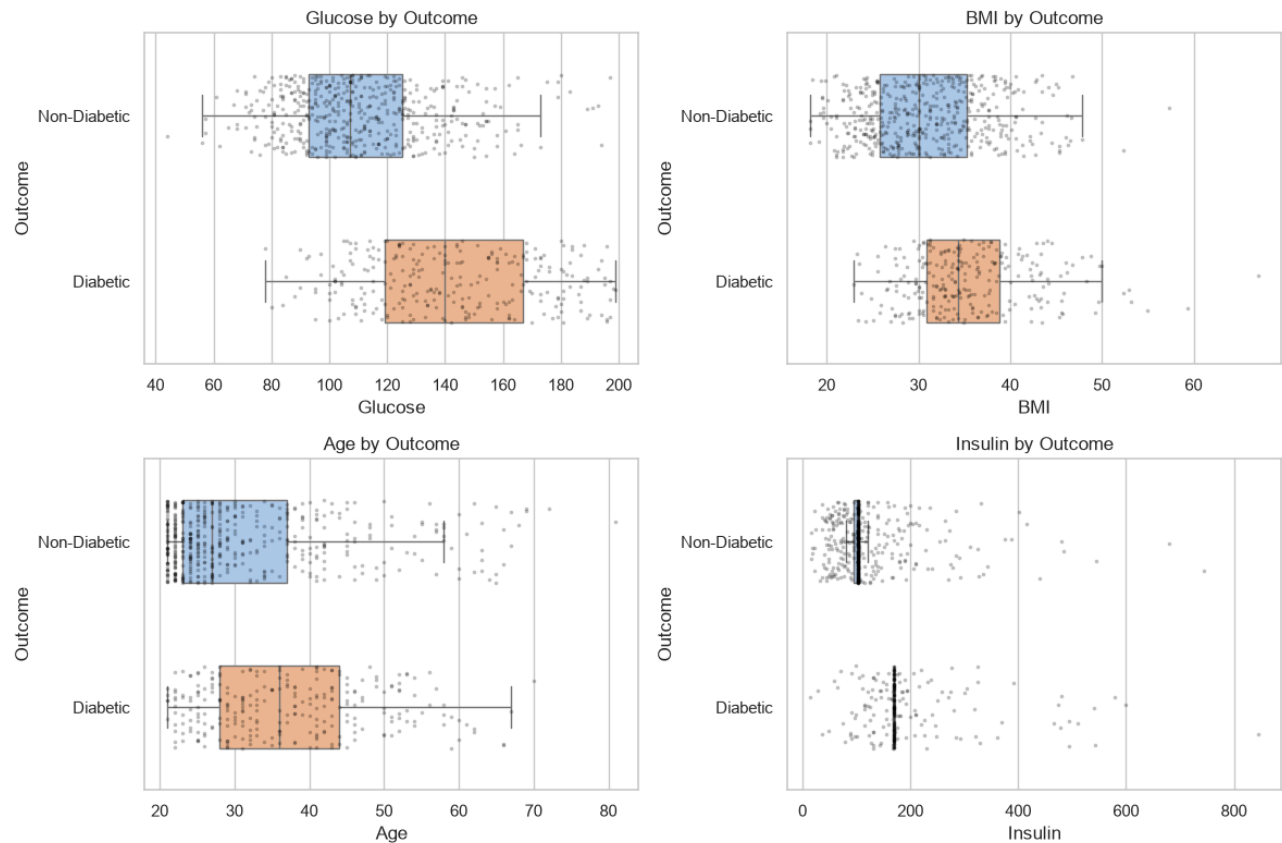
```

box_features = ['Glucose', 'BMI', 'Age', 'Insulin']
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, feature in zip(axes, box_features):
    sns.boxplot(y='Outcome', x=feature, data=df, ax=ax, orient='h', hue='Outcome', palette='pastel', w
    sns.stripplot(y='Outcome', x=feature, data=df, ax=ax, orient='h', color='black', alpha=0.25, size=
    ax.set_yticks([0, 1])
    ax.set_yticklabels(['Non-Diabetic', 'Diabetic'])
    ax.set_title(f'{feature} by Outcome')

plt.tight_layout()
plt.show()

```



Class Distribution of the Target

- **Visualization:** A donut chart representing the proportion of instances in each outcome class.
- **Analytical Purpose:** This confirms the presence of class imbalance in the dataset, justifying the reliance on metrics such as the F1-score and Recall over standard Accuracy.

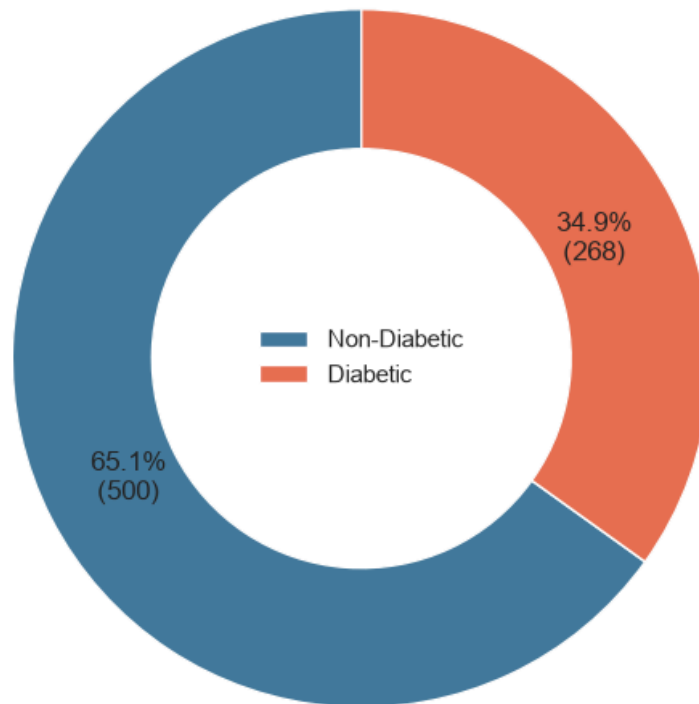
```

counts = df['Outcome'].value_counts().sort_index()
labels = ['Non-Diabetic', 'Diabetic']
colors = ['#457b9d', '#e76f51']

fig, ax = plt.subplots(figsize=(6, 6))
wedges, _, autotexts = ax.pie(
    counts, labels=None, colors=colors, autopct=lambda p: f'{p:.1f}%\n({int(round(p/100*counts.sum())})',
    startangle=90, pctdistance=0.75, wedgeprops={'width': 0.4, 'edgecolor': 'white'})
ax.legend(wedges, labels, loc='center', frameon=False)
ax.set_title('Outcome Class Balance')
plt.tight_layout()
plt.show()

```

Outcome Class Balance



Feature Distributions

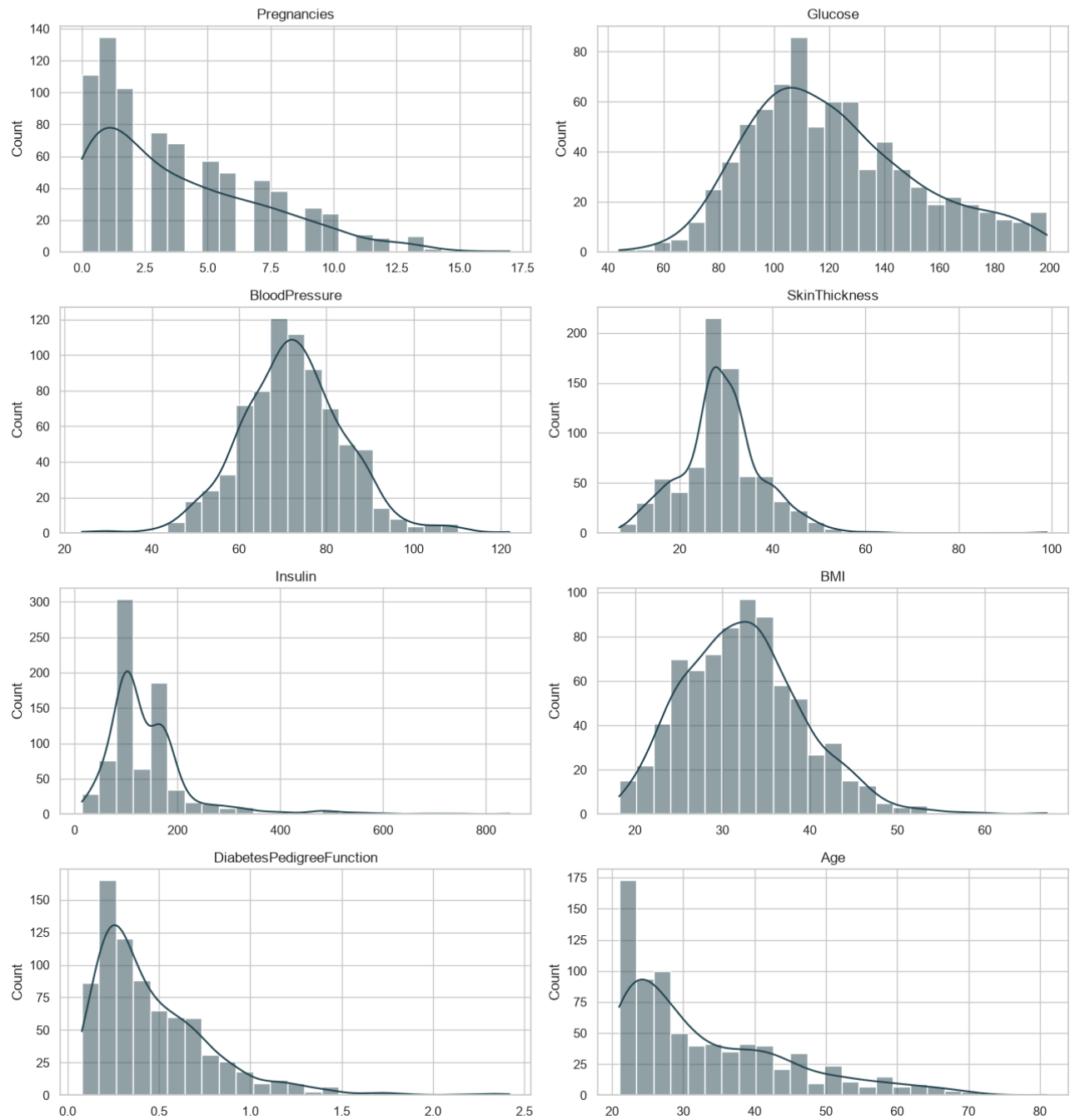
- **Visualization:** Smoothed histograms with kernel density estimates for all input features.
- **Analytical Purpose:** Evaluating the skewness and general shape of feature distributions informs the necessity for preprocessing steps and helps validate the assumptions of certain statistical models.

```
feature_cols = [c for c in df.columns if c != 'Outcome']
fig, axes = plt.subplots(4, 2, figsize=(13, 14))
axes = axes.flatten()

for ax, col in zip(axes, feature_cols):
    sns.histplot(df[col], bins=25, kde=True, color='#264653', ax=ax, edgecolor='white')
    ax.set_title(col)
    ax.set_xlabel('')

plt.suptitle('Distribution of Each Clinical Feature', y=1.01)
plt.tight_layout()
plt.show()
```

Distribution of Each Clinical Feature



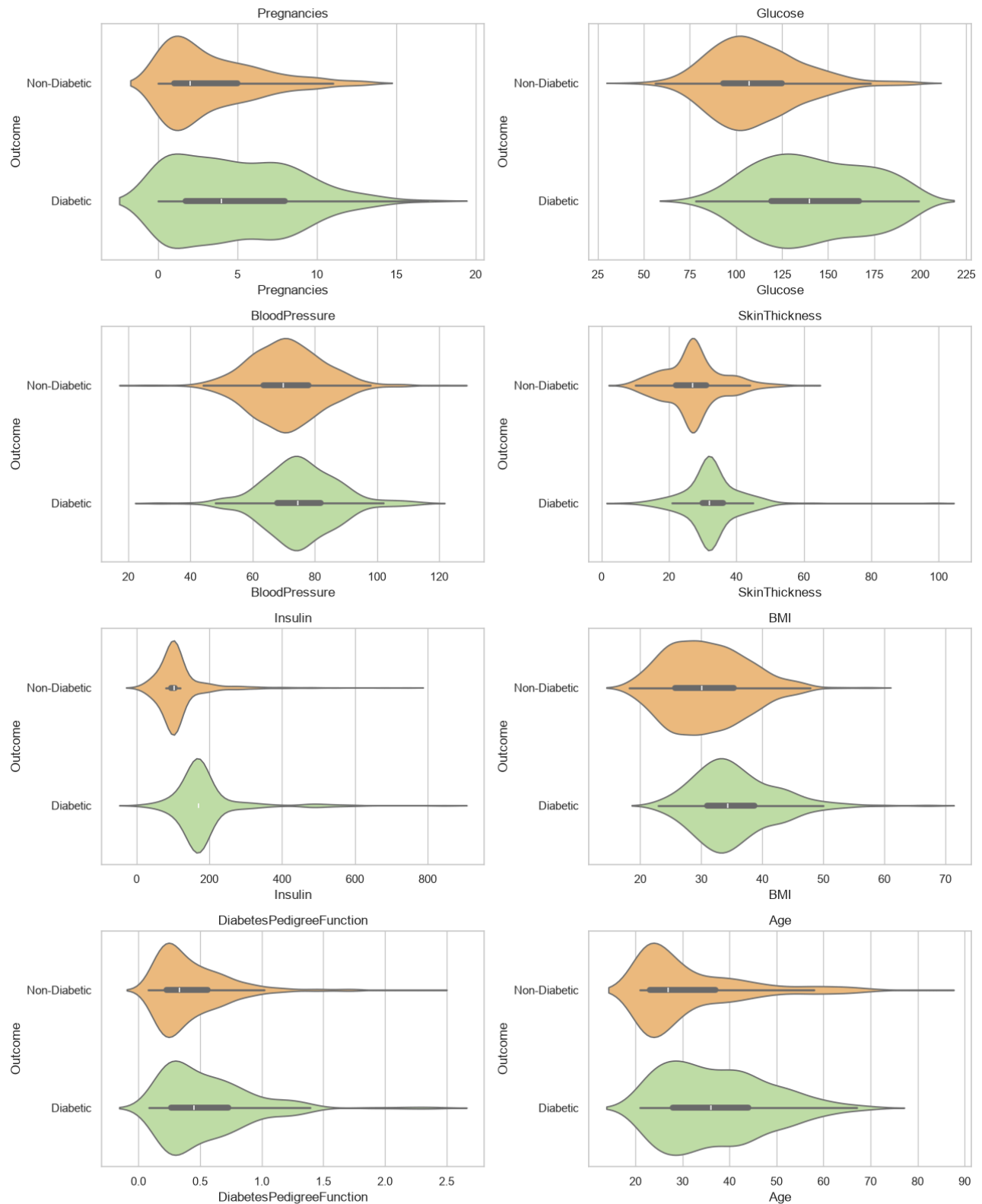
Class-Conditional Feature Distributions

- **Visualization:** Violin plots for all features, partitioned by the target outcome.
- **Analytical Purpose:** By superimposing density estimates over summary statistics, this visualization captures asymmetric or multimodal distributions within each class, offering a more nuanced view than standard box plots.

```
fig, axes = plt.subplots(4, 2, figsize=(13, 16))
axes = axes.flatten()

for ax, feature in zip(axes, feature_cols):
    sns.violinplot(y='Outcome', x=feature, data=df, ax=ax, orient='h', hue='Outcome', palette='Spectral')
    ax.set_yticks([0, 1])
    ax.set_yticklabels(['Non-Diabetic', 'Diabetic'])
    ax.set_title(feature)

plt.tight_layout()
plt.show()
```



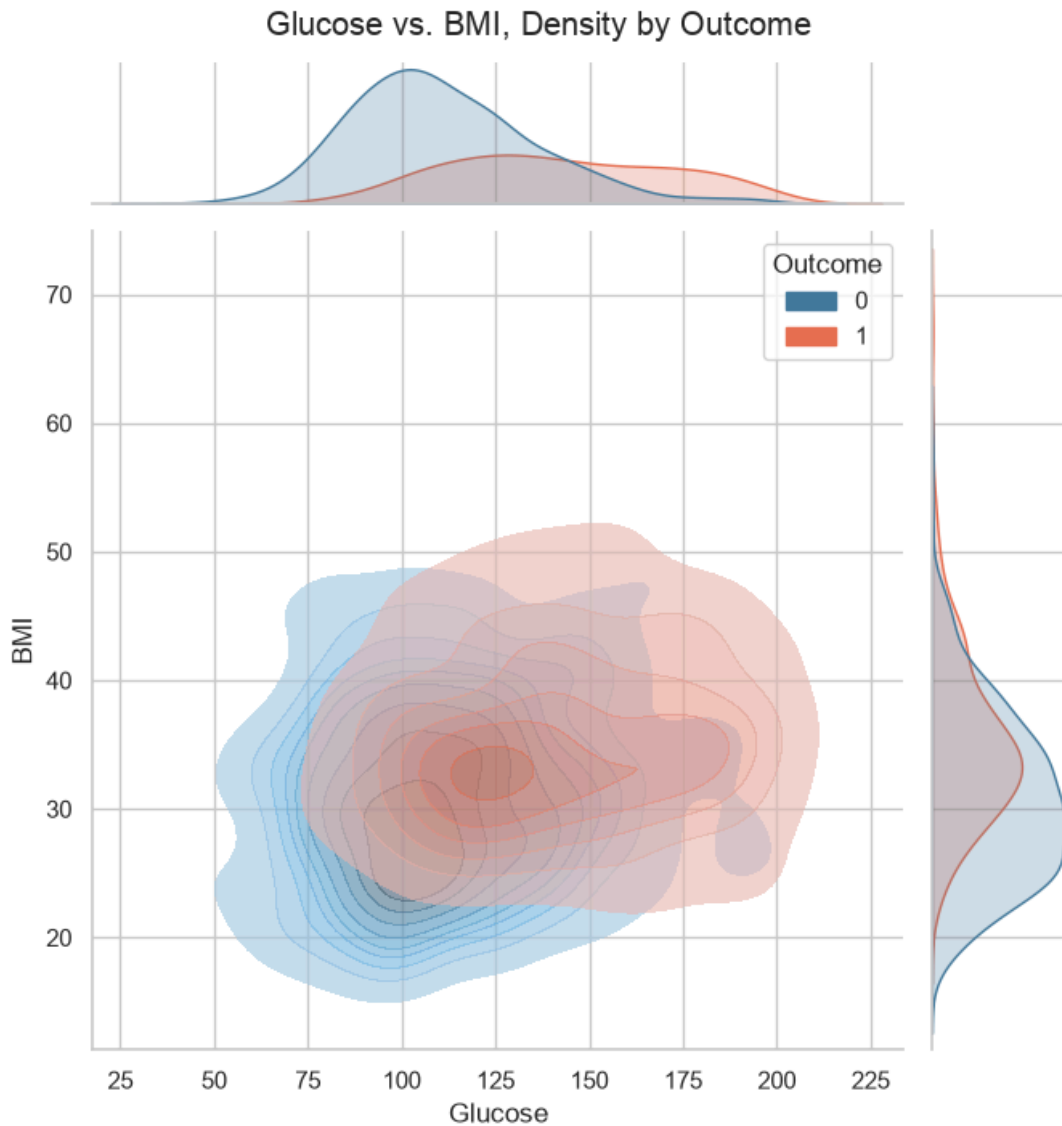
Joint Density of Principal Predictors

- **Visualization:** A bivariate kernel density contour plot mapping Glucose against BMI, color-coded by class outcome.

- **Analytical Purpose:** This analysis illustrates the joint probability density of the two most correlated predictors, highlighting regions of the feature space where the classes are most distinctly separable.

```
df_clean = df[(df['BMI'] > 0) & (df['Glucose'] > 0)]

g = sns.jointplot(x='Glucose', y='BMI', data=df_clean, hue='Outcome', kind='kde',
                  palette={0: '#457b9d', 1: '#e76f51'}, fill=True, alpha=0.6, height=7)
g.fig.suptitle('Glucose vs. BMI, Density by Outcome', y=1.02)
plt.show()
```



Hyperparameter Sensitivity: k-NN

Question. How much does k-NN's performance actually depend on the neighborhood size `k` and the weighting scheme (`uniform` vs. `distance`)?

Setup. Using the cross-validation results already collected during the k-NN grid search, this section looks at how accuracy changes as `n_neighbors` varies over `{3, 5, 7, 9}` and `weights` switches between `uniform` and `distance`. This gives a sense of whether k-NN's performance here is fragile or fairly robust to these choices.

```

knn_cv_df = pd.DataFrame(grid_knn.cv_results_[['param_n_neighbors', 'param_weights', 'mean_test_score']])
knn_cv_df.columns = ['k', 'weights', 'Mean CV Accuracy', 'Std']
knn_cv_df = knn_cv_df.sort_values('Mean CV Accuracy', ascending=False).reset_index(drop=True)
display(knn_cv_df.style.format({'Mean CV Accuracy': '{:.4f}', 'Std': '{:.4f}'}))

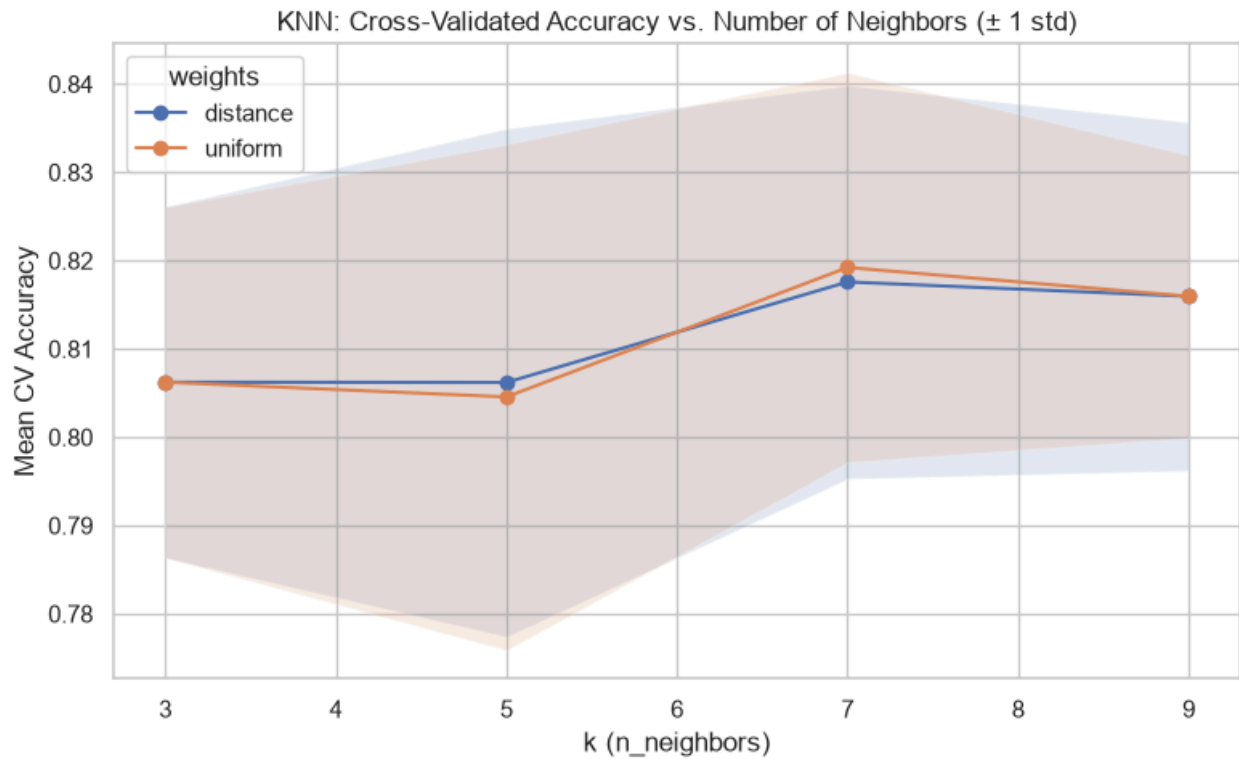
plt.figure(figsize=(8, 5))
for weight_scheme, group in knn_cv_df.groupby('weights'):
    group = group.sort_values('k')
    plt.plot(group['k'], group['Mean CV Accuracy'], marker='o', label=weight_scheme)
    plt.fill_between(group['k'],
                    group['Mean CV Accuracy'] - group['Std'],
                    group['Mean CV Accuracy'] + group['Std'], alpha=0.15)
plt.title('KNN: Cross-Validated Accuracy vs. Number of Neighbors ( $\pm 1$  std)')
plt.xlabel('k (n_neighbors)')
plt.ylabel('Mean CV Accuracy')
plt.legend(title='weights')
plt.tight_layout()
plt.show()

print(f"Best configuration: k={grid_knn.best_params_['n_neighbors']}, weights='{grid_knn.best_params_['weights']}' -> CV accuracy = {grid_knn.best_score_:.4f}")

spread = knn_cv_df['Mean CV Accuracy'].max() - knn_cv_df['Mean CV Accuracy'].min()
print(f"Spread across every k/weights combination tried: {spread:.4f} "
      f"({'small, so k is not a very sensitive hyperparameter here' if spread < 0.02 else 'noticeable,'})

```

	k	weights	Mean CV Accuracy	Std
0	7	uniform	0.8192	0.0220
1	7	distance	0.8176	0.0223
2	9	uniform	0.8160	0.0160
3	9	distance	0.8160	0.0197
4	3	uniform	0.8062	0.0199
5	3	distance	0.8062	0.0199
6	5	distance	0.8062	0.0288
7	5	uniform	0.8046	0.0286



Best configuration: `k=7, weights='uniform'` -> CV accuracy = 0.8192
Spread across every k/weights combination tried: 0.0147 (small, so k is not a very sensitive hyperparameter here).

9. Model Comparison

Question. Which model generalizes best on the held-out test set, taking Accuracy, Precision, Recall, and F1-score together rather than any single metric?

```

results_df = pd.DataFrame(model_metrics)
results_df_display = results_df.copy()
results_df_display['Accuracy'] = results_df_display['Accuracy'] * 100
results_df_display = results_df_display.sort_values(by='F1', ascending=False).reset_index(drop=True)

print("=== Model Comparison, ranked by F1 ===")
display(results_df_display.style.format({
    'Accuracy': '{:.2f}%', 'Precision': '{:.3f}', 'Recall': '{:.3f}', 'F1': '{:.3f}'
}))

metrics_long = results_df_display.melt(id_vars='Model', value_vars=['Accuracy', 'Precision', 'Recall',
                                                                    var_name='Metric', value_name='Score'])
metrics_long.loc[metrics_long['Metric'] != 'Accuracy', 'Score'] *= 100

plt.figure(figsize=(12, 6))
ax = sns.barplot(data=metrics_long, x='Model', y='Score', hue='Metric', palette='crest')
plt.title('Model Performance Across All Metrics')
plt.ylabel('Score (%)')
plt.xlabel('')
plt.xticks(rotation=25, ha='right')
plt.legend(title='Metric', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()

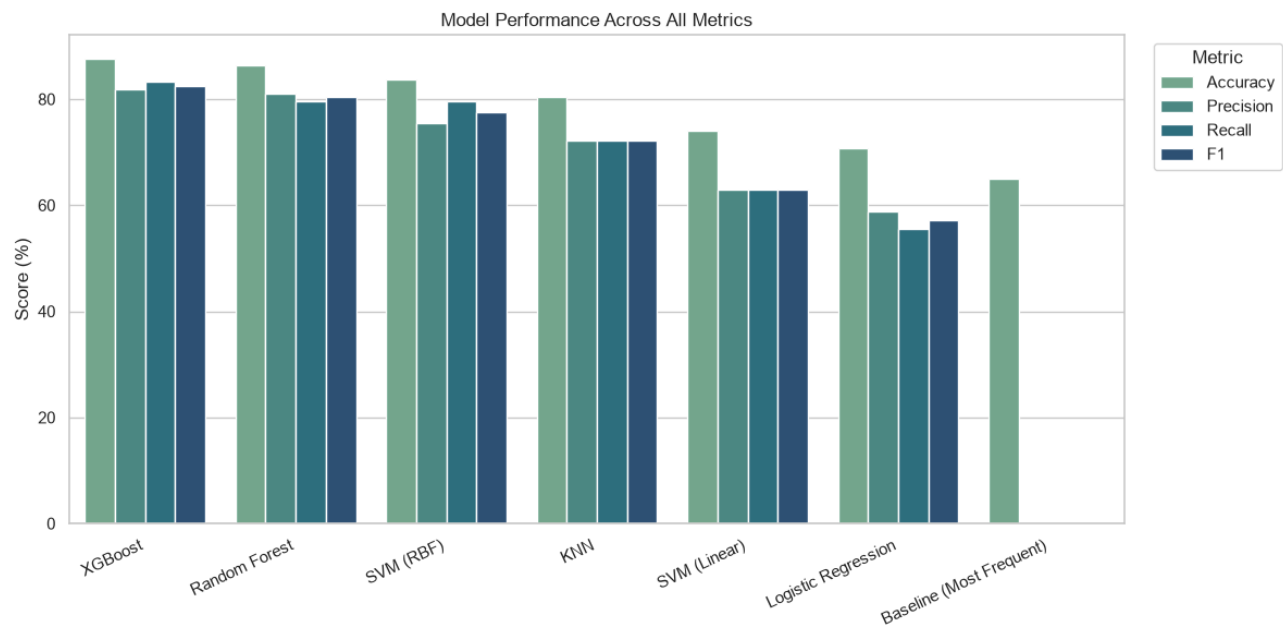
```

```

=== Model Comparison, ranked by F1 ===

```

	Model	Accuracy	Precision	Recall	F1
0	XGBoost	87.66%	0.818	0.833	0.826
1	Random Forest	86.36%	0.811	0.796	0.804
2	SVM (RBF)	83.77%	0.754	0.796	0.775
3	KNN	80.52%	0.722	0.722	0.722
4	SVM (Linear)	74.03%	0.630	0.630	0.630
5	Logistic Regression	70.78%	0.588	0.556	0.571
6	Baseline (Most Frequent)	64.94%	0.000	0.000	0.000



10. Representation Analysis

Question. Does standardizing the feature space actually make a measurable difference for a distance-based model like k-NN?

Setup. k-NN is trained once on the raw, unscaled features and once on the standardized features, and the two are compared directly. This is a direct illustration of how much the choice of representation shapes what a model can learn, independent of the algorithm itself.

```

from sklearn.neighbors import KNeighborsClassifier

knn_unscaled = KNeighborsClassifier(n_neighbors=5)
knn_unscaled.fit(X_train, y_train)
acc_unscaled = accuracy_score(y_test, knn_unscaled.predict(X_test))

knn_scaled = KNeighborsClassifier(n_neighbors=5)
knn_scaled.fit(X_train_scaled, y_train)
acc_scaled = accuracy_score(y_test, knn_scaled.predict(X_test_scaled))

print(f'KNN accuracy on raw (unscaled) features:  {acc_unscaled*100:.2f}%')
print(f'KNN accuracy on standardized features:    {acc_scaled*100:.2f}%')

comparison = pd.DataFrame({'Representation': ['Unscaled', 'Standardized'],
                           'Accuracy': [acc_unscaled * 100, acc_scaled * 100]})
plt.figure(figsize=(5, 4.5))
bars = plt.bar(comparison['Representation'], comparison['Accuracy'], color=['#adb5bd', '#2a9d8f'], width=0.8)
for bar, val in zip(bars, comparison['Accuracy']):
    plt.text(bar.get_x() + bar.get_width() / 2, val + 0.5, f'{val:.2f}%', ha='center', fontweight='bold')
plt.ylim(0, 100)
plt.ylabel('Test Accuracy (%)')
plt.title('KNN Accuracy: Raw vs. Standardized Features')
plt.tight_layout()
plt.show()

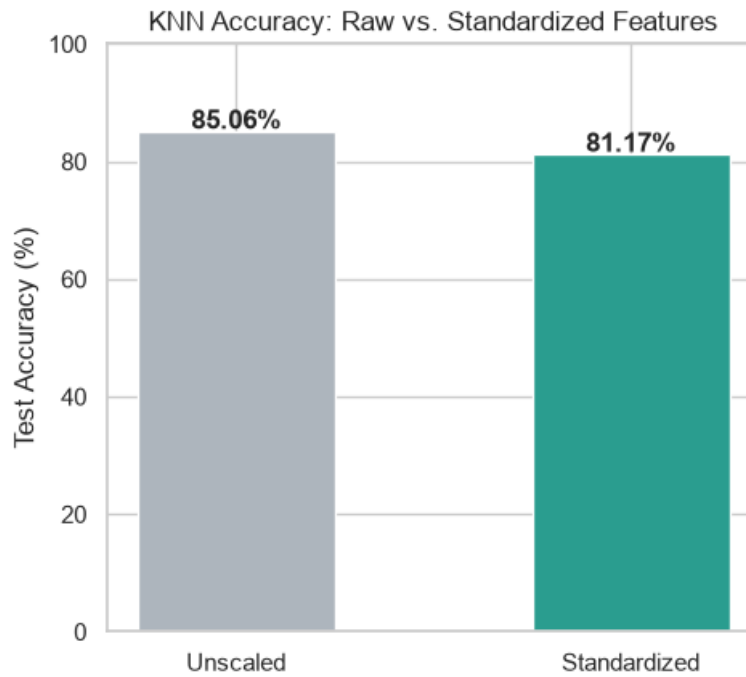
gap = acc_scaled - acc_unscaled
if gap > 0.005:
    verdict = (f"standardizing helped here (+{gap*100:.2f} pts) – features with naturally large ranges "
              "like Insulin would otherwise dominate the distance calculation.")
elif gap < -0.005:
    verdict = (f"on this particular split, the unscaled features actually did slightly better "
              f"({-gap*100:.2f} pts higher) than the standardized ones. That does not overturn the ge "
              "rule that KNN is distance-based and therefore scale-sensitive – it just means this tra "
              "split and value of k happen not to expose the gap. Standardization remains the safer d "
              "for any distance-based model, and is applied throughout the rest of this notebook.")
else:
    verdict = "the two representations performed almost identically on this particular split."
print(f'\nTakeaway: {verdict}')

```

```

KNN accuracy on raw (unscaled) features:  85.06%
KNN accuracy on standardized features:    81.17%

```



Takeaway: on this particular split, the unscaled features actually did slightly better (3.90 pts higher) than the standardized ones. That does not overturn the general rule that KNN is distance-based and therefore scale-sensitive – it just means this train/test split and value of k happen not to expose the gap. Standardization remains the safer default for any distance-based model, and is applied throughout the rest of this notebook.

Final Model Selection

The final model is chosen programmatically from the comparison table: whichever classifier has the highest F1-score on the test set is selected for the demo application. The baseline is excluded from this selection, since it exists only to set a performance floor.

```

candidates_df = results_df_display[results_df_display['Model'] != 'Baseline (Most Frequent)']
best_row = candidates_df.sort_values('F1', ascending=False).iloc[0]
best_model_name = best_row['Model']
best_model_file = model_filenames[best_model_name]
final_model_needs_scaling = model_needs_scaling[best_model_name]
final_model = joblib.load(best_model_file)

print(f"Selected final model: {best_model_name}")
print(f"  -> Serialized as: {best_model_file}")
print(f"  -> Accuracy={best_row['Accuracy']:.2f}%, Precision={best_row['Precision']:.3f}, "
      f"Recall={best_row['Recall']:.3f}, F1={best_row['F1']:.3f}")
print("Selection rule: highest F1 on the held-out test set among the trained models "
      "(F1 balances Precision and Recall, which fits this imbalanced medical-screening task well).")

```

```

Selected final model: XGBoost
  -> Serialized as: xgboost.pkl
  -> Accuracy=87.66%, Precision=0.818, Recall=0.833, F1=0.826
Selection rule: highest F1 on the held-out test set among the trained models (F1 balances Precision and
Recall, which fits this imbalanced medical-screening task well).

```

11. Intelligent Application

To show the trained model working outside this report, the best-performing classifier (see Section 9) is packaged into a small full-stack demo application: a backend that loads the serialized model and preprocessing objects, and a React frontend for entering patient data and viewing the result.

The four screenshots below walk through one full prediction made in the running application, following this pipeline step by step from raw input to explanation.

1. Patient intake. A clinician (or patient) enters the eight clinical measurements. The form starts pre-filled with a sample profile that can be edited before running the assessment.

**GlucoseSense**
ENSEMBLE DIABETES RISK ASSESSMENT

Assess

Batch Upload

History

Model Insights

Patient Intake
Values pre-filled with a sample profile — edit before assessing

PREGNANCIES

2count

Dataset range 0–17, typical ≈ 3

PLASMA GLUCOSE

130mg/dL

Dataset range 44–199, typical ≈ 117

BLOOD PRESSURE

70mm Hg

Dataset range 24–122, typical ≈ 72

SKIN THICKNESS

25mm

Dataset range 7–99, typical ≈ 29

SERUM INSULIN

90μU/mL

Dataset range 14–846, typical ≈ 125

BODY MASS INDEX

28.5kg/m²

Dataset range 18–67, typical ≈ 32

PEDIGREE FUNCTION

0.45score

Dataset range 0–2, typical ≈ 0

AGE

33years

Dataset range 21–81, typical ≈ 29

Run Assessment

Fill in the patient intake form and run an assessment to see the ensemble risk score and per-model breakdown.

Figure 1. Patient intake form, pre-filled with a sample profile.

2. Prediction result. Submitting the form sends the feature vector through the same preprocessing pipeline used during training (imputation, then scaling) before it reaches the model. The app returns a diabetic risk score, a binary classification, and how many of the six trained models agreed on it — here, a 33.9% risk score, classified Non-Diabetic, with 83% ensemble agreement.

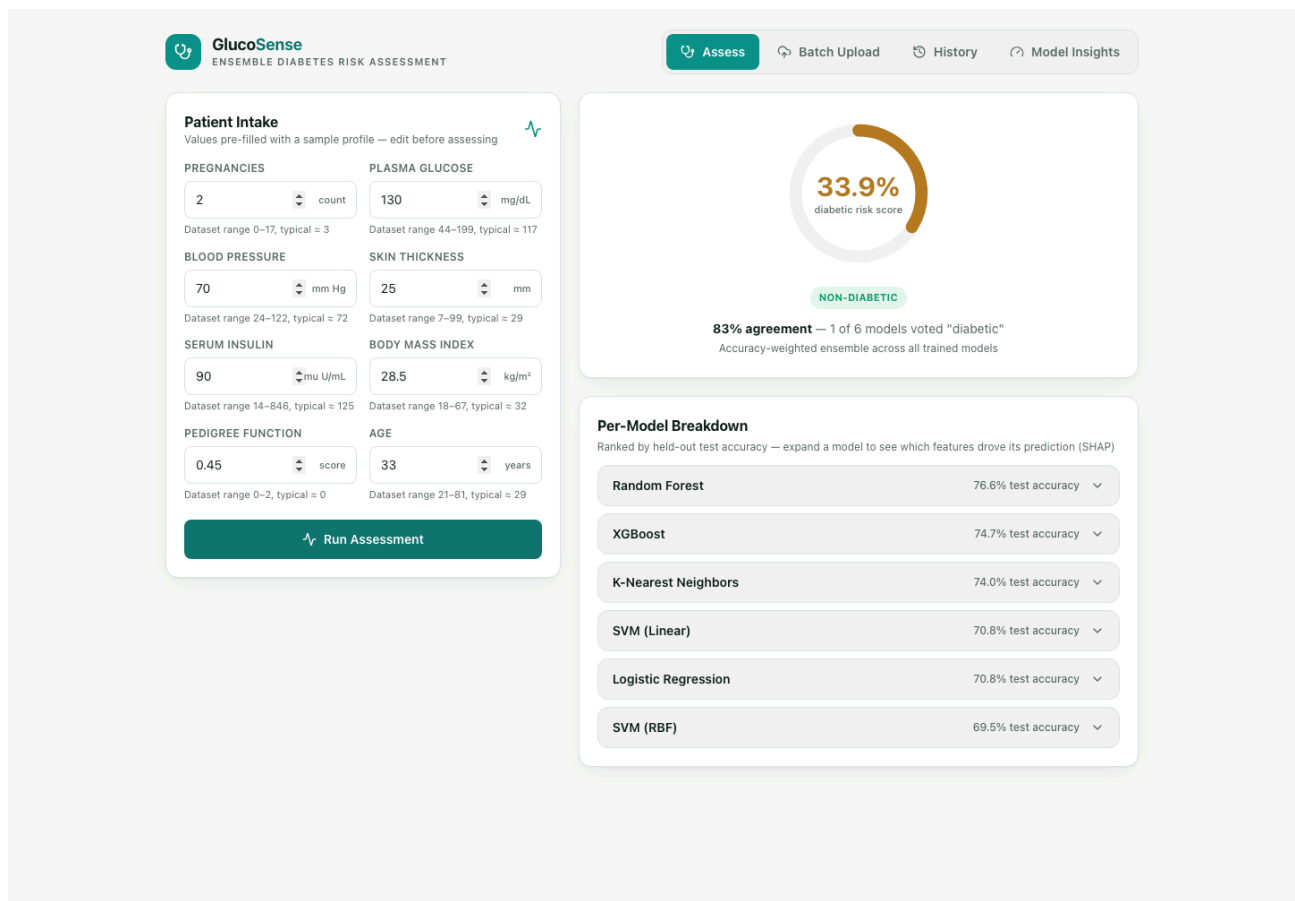


Figure 2. Ensemble risk score and classification returned for the submitted profile.

3. Per-model breakdown. Each model in the ensemble can be expanded individually. Expanding Random Forest shows a SHAP-based explanation of which features pushed its prediction toward or away from "diabetic" for this specific patient — Age, Glucose, and BloodPressure pushed the score up, while BMI and Insulin pulled it down.

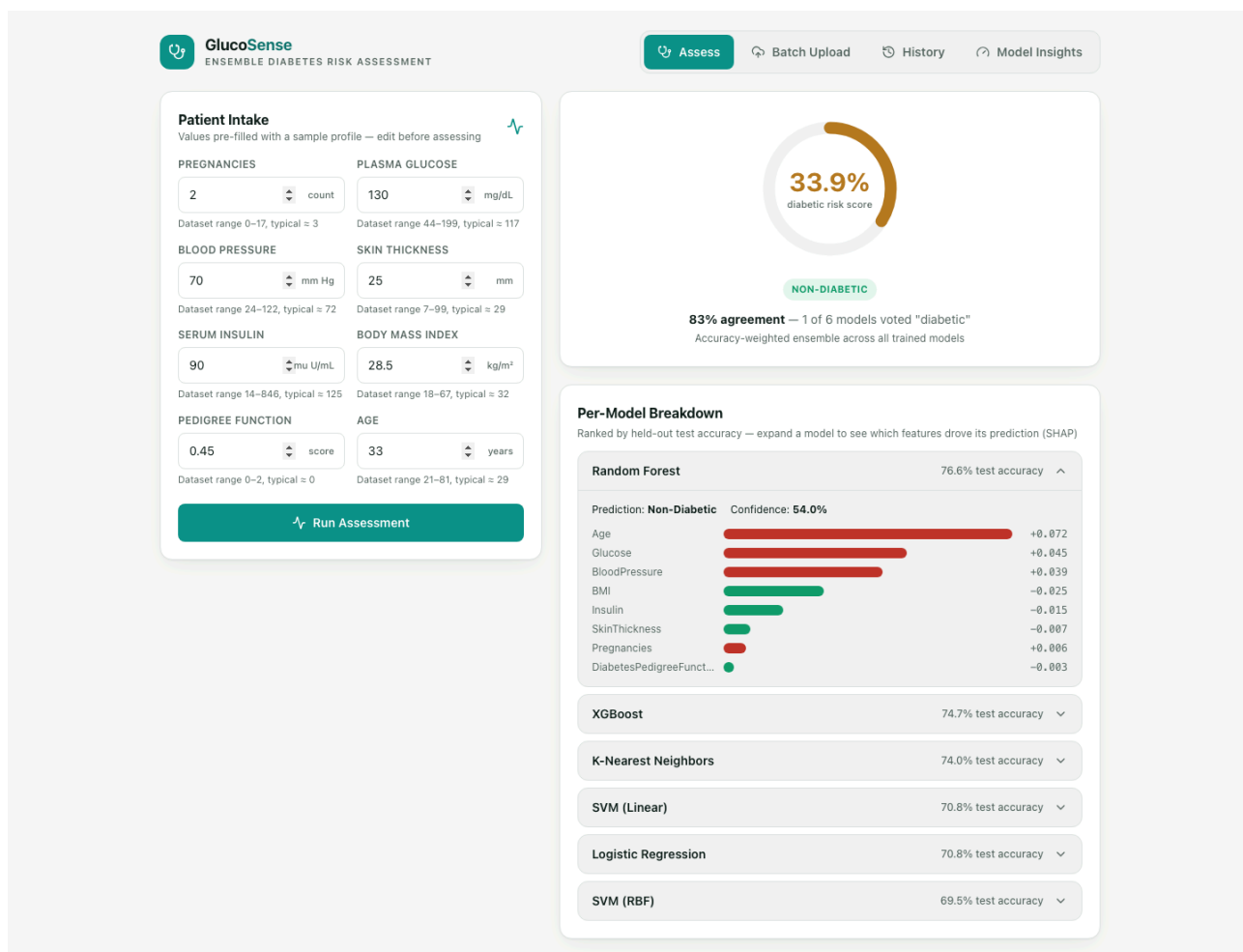


Figure 3. Per-model results with a SHAP feature-contribution breakdown for Random Forest.

4. Model insights dashboard. A separate view aggregates results across the whole test set: a side-by-side comparison of Accuracy/Precision/Recall/F1 for all six models, global feature importance from the Random Forest, and the confusion matrix for every model. This is the same information reported in Sections 8–9, surfaced inside the application itself rather than only in this report.

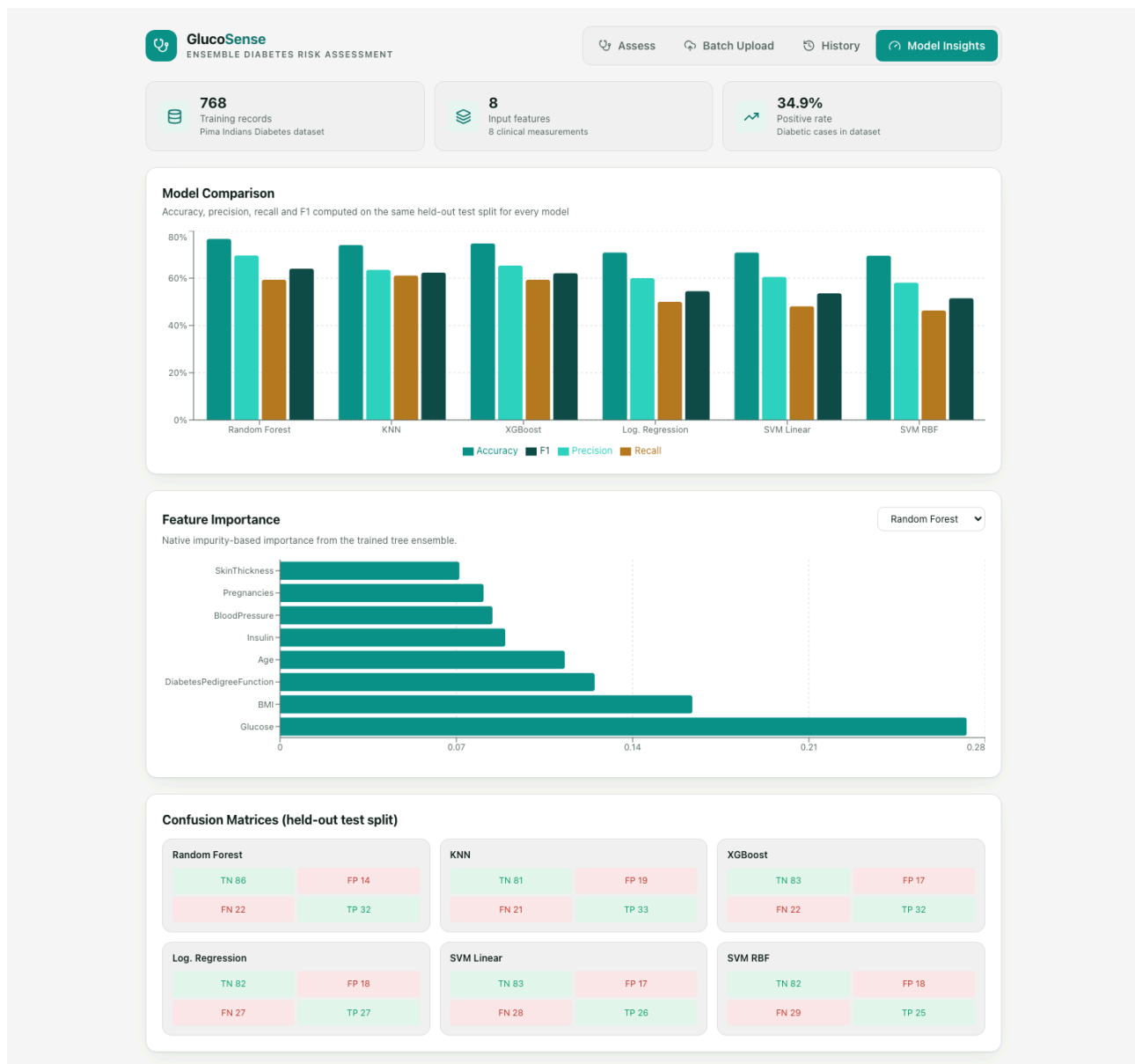


Figure 4. Model comparison, feature importance, and confusion matrices, aggregated across the test set.

12. Limitations

The system shows real predictive ability, but it isn't ready to be used as a standalone clinical tool. A few limitations are worth being upfront about:

- **Population bias.** The training data comes only from female patients of Pima Indian heritage, so the learned decision boundary may not transfer well to patients with different demographics or physiology.
- **No sense of time.** Each feature vector is a single snapshot of a patient's health. The system has no way to account for how a patient's condition is changing over time.
- **Imputation is a simplification.** Replacing invalid zero readings with the training median is a reasonable default, but it can understate the real uncertainty in incomplete clinical data and slightly flattens feature variance.
- **Small dataset.** With only 768 records, the reported metrics carry a fair amount of sampling variance and should be read as estimates, not guarantees of real-world performance.

- **Screening tool, not a diagnosis.** This system is meant to support an initial triage decision. Its output is a probability estimate, not a medical diagnosis, and should always be combined with clinical judgment.

13. Reflection

Even though it's built from classical ML rather than anything fancier, this system does show the basic ingredients of an intelligent system: it turns raw environmental data into a structured representation, finds statistical patterns in that representation through supervised learning, and uses those patterns to make decisions about new, unseen patients.

The 8-dimensional feature vector is a reasonable representation for this problem — it captures the relevant physiological measurements in a form that classical ML models can use directly. That said, a flat feature vector like this throws away any spatial or temporal structure that a richer form of medical data might have, such as imaging or a time series of visits. Representing that kind of data well would call for a different approach entirely — tensors for images, sequences for models like RNNs.

Running the same preprocessing, training, and evaluation pipeline across seven different models made it easy to compare them on equal footing, and the results made it clear just how much the combination of representation and algorithm matters — the same data produced fairly different performance depending on whether the model was linear, distance-based, or tree-based, and whether the features were scaled at all.

14. Conclusion

This assignment goes through the full process of building a classical machine learning system, from defining a real problem to shipping a working demo that uses the trained model. The results reinforce a point that's easy to underestimate: how the data is represented — in particular, whether distance-sensitive models get standardized input — has as much impact on performance as the choice of algorithm itself, and careful hyperparameter tuning is what turns a decent model into a reliably good one. Overall, the finished system is a working example of the core idea behind intelligent systems: taking raw environmental data, shaping it into a representation a model can learn from, and using that to make useful decisions.